Dynamic Speculative Agent Planning

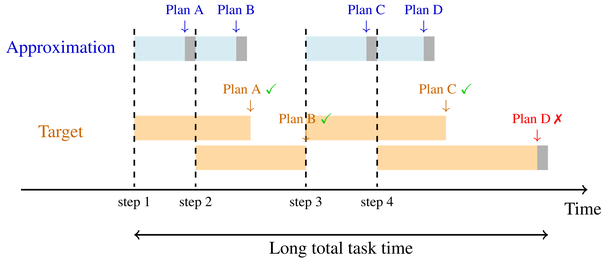
Johns Hopkins University · Microsoft Research · UCSB Santa Barbara · Google DeepMind

Yilin Guan Qingfeng Lan Fei Sun Dujian Ding
Devang Acharya Chi Wang William Yang Wang Wenyue Hua

Paper Code Personal Site

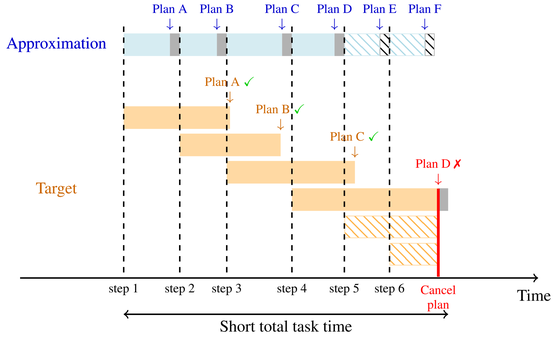
Why Dynamic?

Problem with **fixed-k** speculative planning



(a) speculative step too small ($k = 2$).

Too few speculative steps: insufficient speedup.



(b) speculative step too large ($k = 6$).

Too many speculative steps: wastes cost if the verifier rejects.

How we trained our predictor:

Optimal k: the number of speculative steps that the approximation agent can generate correctly from current state before first mismatch.

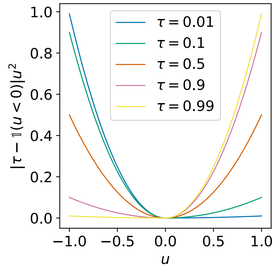
Current Trajectory → Predictor Estimates Optimal Speculative Steps (K)

Cheap Agent Approximates:	Step 1	Step 2	Step 3
Strong Agent Verifies:	Step 1 ✅	Step 2 ✅	Step 3 ❌
Rewards:	+1	+1	0

Return = (+1) + (+1) + 0 = 2

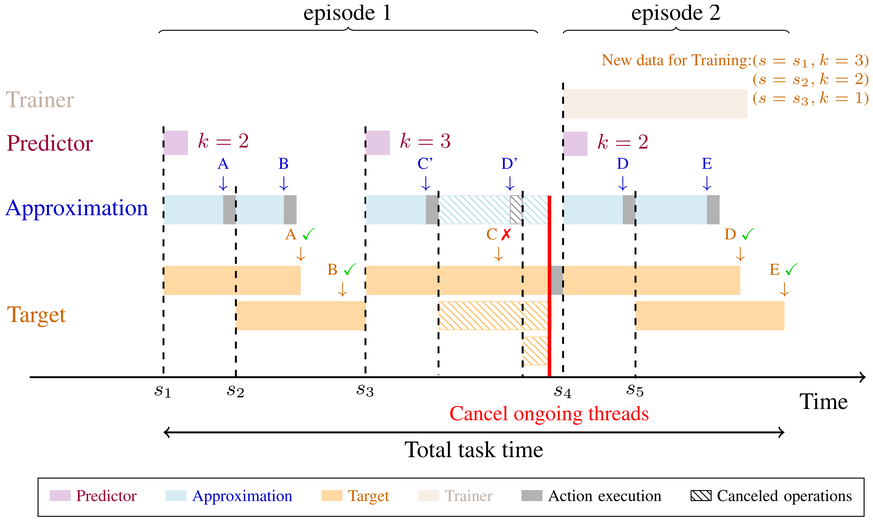
Small predictor (DistillBERT) learns to predict the optimal speculative steps k, which gives maximum speedup without invalid approximations.

Trained Async - background thread, never blocks the planning pipeline.



Training-time bias control through τ

- Low τ (conservative k): cheaper, slower
- High τ (aggressive k): costlier, faster

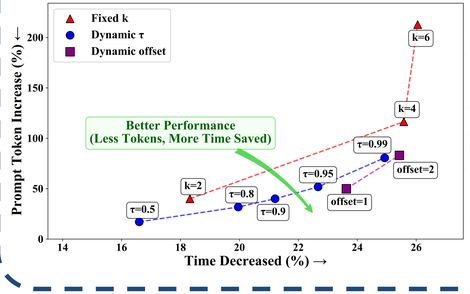


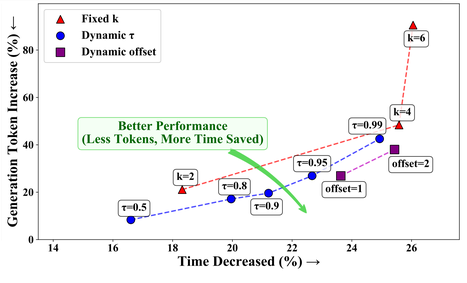
Our Method DSP

1. Predict k from the current trajectory
2. Run approximation and target in parallel
3. Cancel on mismatch; train asynchronously online

Table 1: Dynamic k vs. Fixed k Strategies on **OpenAGI** benchmark with **GPT** backbone.

Mode	Direct-ReAct (Setting 1)						CoT-MAD (Setting 2)					
	T (×)	P (×)	G (×)	Cost (×)	$\overline{MC}$	$\overline{K}$	T (×)	P (×)	G (×)	Cost (×)	$\overline{MC}$	$\overline{K}$
Fix (k = 2)	1.00	1.00	1.00	1.00	3.00	2.00	1.00	1.00	1.00	1.00	3.00	2.00
Fix (k = 4)	0.92	1.56	1.23	1.41	4.95	4.00	0.83	1.41	1.38	1.37	4.97	4.00
Fix (k = 6)	**0.90**	2.24	1.57	1.92	6.32	6.00	0.81	1.87	1.75	1.77	6.44	6.00
Dyn. (τ=0.5)	1.01	0.87	0.91	**0.88**	4.33	1.78	0.98	0.95	0.95	**0.95**	4.44	2.10
Dyn. (τ=0.8)	0.97	0.97	0.98	0.97	4.41	2.33	0.89	1.04	1.04	1.04	4.73	2.67
Dyn. (τ=0.9)	0.95	1.04	1.00	1.02	4.56	2.47	0.86	1.12	1.11	1.11	5.22	3.03
Dyn. (τ=0.95)	0.94	1.12	1.07	1.09	4.75	2.88	0.85	1.26	1.25	1.24	5.64	3.58
Dyn. (τ=0.99)	0.91	1.32	1.19	1.25	5.24	3.59	0.82	1.41	1.37	1.37	5.87	4.06
Dyn. (offset=1)	0.92	1.10	1.06	1.08	4.86	2.75	0.88	1.11	1.09	1.10	4.95	2.80
Dyn. (offset=2)	**0.90**	1.33	1.15	1.25	5.13	3.47	**0.80**	1.29	1.27	1.27	5.60	3.99





DSP achieves a better **latency–cost Pareto frontier** than fixed-k baselines

- Up to **30%** lower total cost
- Up to **60%** lower unnecessary cost
- **40%** lower sustained system pressure